AUTONATEAI SCREEPS LEAGUE

Tutorial 10: AI Collaboration

A practical local-server Screeps module **docs/tutorials/10-ai-collaboration.md**

Episode 10: Two Minds

"You've been working with me this whole time," your collaborator says, "but you've never had to think about *how* you were working with me. That's the next skill. Not writing better Screeps code — writing better prompts about your Screeps code. There's a version of you that's been shipping for a decade and still writes 'it's broken, pls help' in every bug report. Don't be that version."

Nine episodes of debugging happened mostly by instinct. This one makes the instinct repeatable.

Goal

Build a deliberate practice around AI collaboration instead of an accidental one:

- Learn the shape of a prompt that actually gets a useful answer
- Break something in your own codebase on purpose and fix it using that shape
- Build a checklist for reviewing AI-suggested code before it runs on a live colony
- Start a learning journal so this debugging session becomes next season's curriculum

Prerequisites

Tutorial 09 is complete:

- `cache.js`, `role.hauler.js`, `role.upgrader.js`, and `role.builder.js` are all working together.
- You have a CPU log printing every 20 ticks.

Step 1: The Anatomy of a Useful Prompt

A prompt that gets a fast, correct answer about Screeps code usually has five things in it:

- **What you expected** — the behavior you thought the code would produce.
- **What actually happened** — the observed behavior, an error code, or a console log line.
- **The relevant code** — the specific function or block, not the whole file.
- **What you already tried** — so the answer doesn't repeat a dead end.
- **What "done" looks like** — the checkpoint that tells both of you the fix worked.

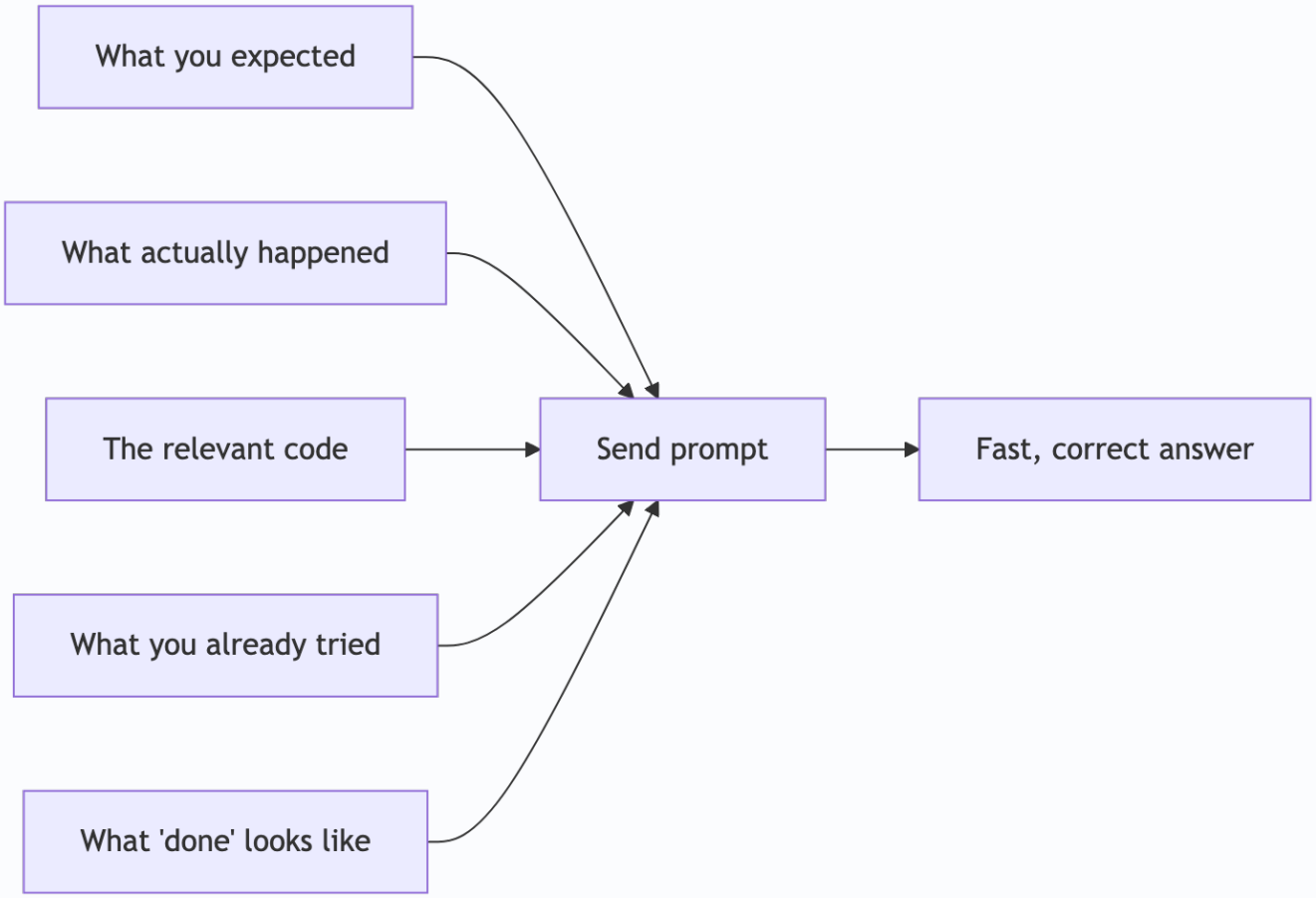
Compare:

> "my hauler is broken help"

against:

> "My haulers in `role.hauler.js` aren't delivering energy — they sit next to the spawn full of energy and don't transfer. Here's the toggle logic: `[code]`. I expected a full hauler to switch to delivery mode. Instead nothing happens. I haven't changed the delivery-target logic, only the toggle condition above it recently. What's wrong with the toggle?"

The second one is longer to write and faster to get right. That trade is almost always worth it.



Skip any one of the five and the answer usually still arrives – it's just more likely to be a plausible guess instead of a correct diagnosis.

Step 2: Break Something on Purpose

Open `role.hauler.js`. Find this line:

```
if (!creep.memory.hauling && creep.store.getFreeCapacity() === 0) {
```

Change it to:

```
if (!creep.memory.hauling && creep.store.getFreeCapacity() > 0) {
```

Save it. This inverts the toggle – a hauler will now think it's full and ready to deliver *before* it's picked anything up, instead of after.

Checkpoint: haulers sit near containers or near the spawn, full or empty, not doing much of anything useful. No error appears in the console. This bug is silent on purpose – `creep.transfer()` on an empty creep just returns `ERR_NOT_ENOUGH_RESOURCES` and does nothing. Nothing crashes. The colony just quietly stops moving energy.

Step 3: Debug It Using Step 1's Shape

Write your own prompt to your AI collaborator using the five-part structure from Step 1, based on what you're actually observing (not by copying the example – describe what you see). Ask it to find the bug.

Checkpoint: your collaborator should be able to point at the inverted comparison without you telling it what you changed, if your prompt included enough of the surrounding logic and an accurate description of the symptom. Fix the line back to `=== 0` and confirm haulers resume delivering.

Step 4: The Pre-Flight Checklist

Before you paste any AI-suggested change into a live colony's code, run it through this list:

- Does it use real Screeps constants (`RESOURCE_ENERGY`, `STRUCTURE_EXTENSION`) instead of string literals it invented (`'energy'`, `'extension'`)?
- Does it handle a `find()` or `findClosestByPath()` call that might return `null` or an empty array?
- Does it match this codebase's existing shape — a role module with `run(creep)`, dispatch through `main`, memory toggles like the ones you've already built?
- Does it write anything new into `Memory` that the dead-creep cleanup loop won't ever remove?
- Would it still work correctly after a global reset, or does it assume something that only a plain cache object (Episode 9) should assume?

An AI collaborator is fast and often right. It has also never seen your specific colony's history, your specific room's terrain, or the reason you wrote `getAssignedSource` the way you did in Episode 2. Verifying against your own codebase's patterns is your job, not something to skip because the suggestion looks confident.

Step 5: Start a Journal

Copy `docs/journal/TEMPLATE.md` to a new file for today, and fill it in based on the exercise you just ran in Steps 2–3. `docs/journal/00-example-entry.md` shows a worked example if you want a model to compare against — don't edit that one, it's a reference, not your log.

The habit matters more than any single entry. A debugging session you don't write down teaches you once. A debugging session you write down teaches whoever reads the journal next — including a future version of you who hit something similar and forgot how it got fixed the first time.

Troubleshooting

If your collaborator's suggested fix doesn't match Step 3's actual bug, check whether your prompt included the real code you changed or a paraphrase of it — paraphrasing a bug description is one of the most common ways to get a plausible-sounding wrong answer.

If you're not sure whether an AI suggestion passes the Step 4 checklist, that uncertainty is the signal to test it in a low-stakes way first — a single creep, a console one-liner, a `console.log` before committing to a full role rewrite.

Completion Criteria

You are done when:

- You've written and sent at least one structured prompt using the Step 1 shape.
- The Step 2 bug is found and reverted.
- You can recite the Step 4 checklist without looking at it.
- A journal entry exists in `docs/journal/` for this session.

Learning Notes

After completing the tutorial, write down (in your journal, not just here):

- What was the actual prompt you sent, word for word?
- Did your collaborator's first answer need a follow-up question, or did it land on the first try?
- Which item on the Step 4 checklist have you actually skipped before, in an earlier episode, without realizing it?
- What's one AI-suggested change you accepted too quickly earlier in this season? What would you do differently now?

Next: Episode 11 — Scouting the Dark

Every episode so far has been about a room you already control. That's about to change.

"You reserved a room in Episode 8 without really looking at it first," your collaborator says. "That worked because it was empty. It won't always be — same energy as signing a lease on a spot you drove past once, at night, with your high beams on."

See `docs/roadmap.md` for the full season.